TRB-014 — Discovery

Submitter: **Mark Brandon**

Request: *Cross-account data exposure: Submission Status queue shows other users' submissions*

What is proposed

A two-phase rollout that closes the cross-account read leak in the Submission Status queue without disrupting the running system. The root cause is two compounding gaps: the Clerk session token sent to Supabase is not a Supabase-verifiable JWT (`src/lib/auth-context.tsx:32` calls `session?.getToken()` with no template), so every request authenticates as `role=anon`; and the scaffolding policies `temp_anon_select_*` / `temp_anon_insert_*` / `temp_anon_update_*` / `temp_anon_delete_*` on `submissions`, `invoices`, `customers`, `submission_invoices`, `proof_of_payment`, and `submitter_profiles` still grant `USING (true)` to `{anon}`. Combined, every authenticated browser request matches the wide-open anon policies and can read any row.

Phase 1 — wire the JWT (zero blast radius). Confirm the Clerk ↔ Supabase integration is live in both dashboards: either native Third-Party Auth (Supabase → Authentication → Sign In/Providers → Third-Party Auth → Clerk registered with the project's Clerk Frontend API URL; Clerk → Integrations → Supabase **Active**), or the legacy path with a Clerk JWT template named exactly `supabase`, signed with the Supabase JWT secret, claims `{ "aud": "authenticated", "role": "authenticated" }`. Update `src/lib/auth-context.tsx:32` to return the correct token for whichever path is active (`session?.getToken()` for native third-party auth, or `session?.getToken({ template: 'supabase' })` for the template path). Existing `temp_anon_*` policies remain in place, so production behavior is unchanged.

Smoke test in production before Phase 2. Sign in as a real user, open DevTools, and run `await supabase.from('submissions').select('count', { head: true })`. Confirm the network request carries `Authorization: Bearer ` and that `SELECT auth.role()` returns `authenticated`, not `anon`. The existing `authenticated_select_own` policies (bound to `{authenticated}`, filtering `user_id = requesting_user_id()`) and the `requesting_user_id()` function reading `request.jwt.claims->>'sub'` will then start enforcing per-user reads. If `auth.role()` stays `anon`, halt — do not proceed to Phase 2.

Phase 2 — drop the anon backstop (only after Phase 1 verified). Remove the scaffolding policies one table at a time, smoke-testing the consuming page after each drop:

```sql
DROP POLICY "temp_anon_select_submissions" ON submissions;
DROP POLICY "temp_anon_insert_submissions" ON submissions;
DROP POLICY "temp_anon_update_submissions" ON submissions;
DROP POLICY "temp_anon_delete_submissions" ON submissions;
```

Repeat for `invoices`, `customers`, `submission_invoices`, `proof_of_payment`, and `submitter_profiles`.

Add defense-in-depth `.eq('user_id', user.id)` filters to both queries in `src/pages/SubmissionDetail.tsx` (the `fetchSubmission` call near line 124 and the 3-second poll near line 86), so the by-id path is double-protected by RLS and explicit filter.

Out of scope (file separately): the `invoices`, `bulk-invoices`, and `screenshots` storage buckets grant SELECT to `{anon, authenticated}` with no user-path scoping — same class of leak for PoP PDFs, invoice PDFs, and step screenshots, but a distinct fix surface.

Desired outcome

Every authenticated user sees only their own customers, invoices, submissions, submission-invoice links, proofs of payment, and submitter profile in the UI — enforced at the Postgres tier by RLS keyed on the Clerk `sub` claim, not by frontend filters alone. Robin Carr (`user_3D5MPxbDE3HwkQKRME4cGRUH6nf`) sees only Auxilium Drywall data in her Submission Status queue; mb@innovagent.ai and Jason Freeman's prior submissions are unreachable from her session. Direct cross-user reads via the supabase-js client, by-id lookups in `SubmissionDetail`, and the 3-second polling loop all return zero rows (or a not-found) when the requesting JWT's `sub` does not match the row's `user_id`. The Clerk ↔ Supabase JWT path is verified in production, the `temp_anon_*` scaffolding policies are gone, and the only remaining SELECT/INSERT/UPDATE/DELETE policies on user-scoped tables are the `authenticated_*_own` family bound to `{authenticated}`.

Success criteria

1. In a live browser session, `await supabase.from('submissions').select('count', { head: true })` shows `Authorization: Bearer ` with `role: "authenticated"` in the decoded payload, and `SELECT auth.role()` returns `authenticated`.
2. As Robin Carr's user, `SELECT count(*) FROM submissions` returns only Auxilium Drywall rows; the Submission Status queue in the UI shows zero submissions belonging to other `user_id`s.
3. As Robin Carr's user, navigating directly to `/submissions/` for a submission owned by mb@innovagent.ai or Jason Freeman returns no row (404 / not-found state), both from the initial `fetchSubmission` and from the 3-second poll.
4. `SELECT policyname FROM pg_policies WHERE tablename IN ('submissions','invoices','customers','submission_invoices','proof_of_payment','submitter_profiles')` returns no rows whose name starts with `temp_anon_`.
5. The full submission flow — create customer → upload invoice → submit → view status → archive — completes end-to-end for a freshly signed-up test user, with no regressions in Dashboard, SubmissionStatus, SubmissionsArchive, Invoices, or `useSubmissionPolling`.
6. `src/pages/SubmissionDetail.tsx` lines ~86 and ~124 both include `.eq('user_id', user.id)` in addition to `.eq('id', id)`.
7. Robin Carr confirms in writing that her queue shows only her own submissions after the fix ships.